Plataforma de torneos de eSports.

Proyecto DAM 1º

Zaloria eSPORTS Tournaments

Zack Herrera

Github: Zack1610

1. Descripción del Proyecto 2

2. Tecnologías Utilizadas 2
3. Explicación de la Arquitectura 2
Capa de Modelo (Entidades) 3
Capa de Controlador 3
Capa de Repositorio 3
4. Instrucciones de Despliegue 3
Requisitos Previos 3
Pasos para la Instalación 4
Puntos clave 4
Anexos Técnicos (Diagramas) 4

1. Descripción del Proyecto

Zaloria eSports es una plataforma integral de gestión de torneos competitivos orientada a la comunidad gamer. El sistema permite automatizar el ciclo de vida de un torneo, desde el reclutamiento de aspirantes hasta la generación de estadísticas históricas.

El proyecto se divide en dos grandes áreas:

- **Panel Administrativo:** Permite gestionar equipos, jugadores, procesar solicitudes de ingreso, iniciar torneos y simular resultados mediante un motor aleatorio que calcula eliminaciones y puntajes.
- **Ecosistema Público:** Una interfaz para fans que incluye noticias de la comunidad, un formulario de inscripción para nuevos talentos y un **Hall of Fame** con rankings dinámicos de equipos (por copas) y jugadores (por eliminaciones acumuladas).

2. Tecnologías Utilizadas

El proyecto se basa en un stack tecnológico moderno de Java empresarial:

- **Lenguaje:** Java 21.
- **Framework Principal: Spring Boot 3** (Ecosistema Spring MVC).
- **Persistencia de Datos: Spring Data JPA** con Hibernate para el mapeo objeto-relacional (ORM).
- **Base de Datos: MySQL 8.0** para el almacenamiento relacional.
- **Motor de Plantillas: Thymeleaf** para renderizar vistas HTML dinámicas desde el servidor.
- **Estilos y UI: Bootstrap 5.3** combinado con CSS personalizado para una estética "Cyberpunk/Gamer".
- **Gestión de Dependencias:** Maven.

3. Explicación de la Arquitectura

El sistema sigue el patrón de diseño **MVC (Modelo-Vista-Controlador)**, organizado de la siguiente manera:

Capa de Modelo (Entidades)

Define la estructura de los datos y sus relaciones en la base de datos:

- **Equipos:** Entidad raíz con atributos como logoUrl y managers.
- **Jugador:** Relacionado de forma "Muchos a Uno" (@ManyToOne) con Equipos.
- **Torneo:** Gestiona estados (ABIERTO, EN CURSO, FINALIZADO) y posee una relación "Muchos a Muchos" (@ManyToMany) con Equipos mediante una tabla intermedia llamada inscripcion.
- **Resultado:** Entidad estadística que vincula un Torneo con su Ganador y MVP, almacenando datos calculados como eliminaciones y puntosTotales.
- **Solicitud:** Gestiona los registros de nuevos usuarios con estados de aprobación.

Capa de Controlador

Gestiona la lógica de negocio y el flujo de navegación:

- **SolicitudController:** Procesa el reclutamiento y la transición de aspirante a jugador oficial.
- **TorneoController:** Contiene el "Escudo de Seguridad" que impide iniciar torneos con menos de 2 equipos.
- **ResultadoController:** Alberga el motor de simulación que elige aleatoriamente ganadores y MVPs.
- **PublicController:** Realiza consultas de agregación SQL (COUNT y SUM) para generar los rankings del Hall of Fame.

Capa de Repositorio

Interfaces que extienden de JpaRepository. Destaca el uso de **JPQL** y **Proyecciones (DTOs)** para realizar consultas complejas de agrupación (GROUP BY) que calculan los podios en tiempo real sin necesidad de tablas adicionales.

4. Instrucciones de Despliegue

Requisitos Previos

1. Java JDK 17 o superior instalado.
2. MySQL Server funcionando.
3. IDE (NetBeans, IntelliJ o VS Code) con soporte para Spring Boot.

Pasos para la Instalación

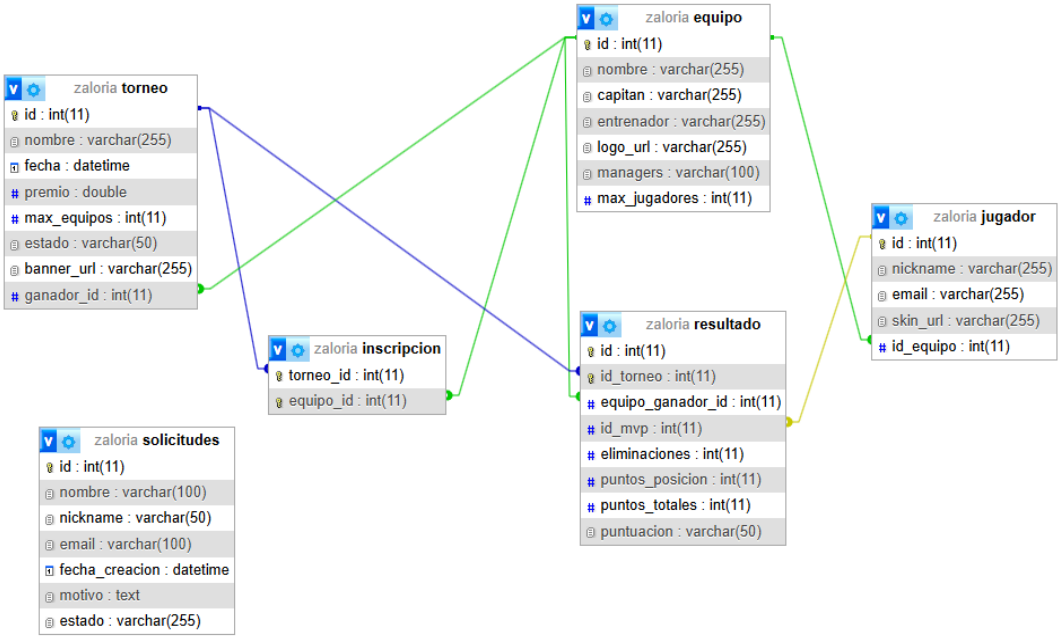
1. **Configurar la Base de Datos:**
    a. Crear una base de datos llamada zaloria.
    b. Ejecutar el script DDL (tablas) e insert de datos básicos para tener equipos y jugadores iniciales.
2. **Configurar el Proyecto:**
    a. En el archivo src/main/resources/application.properties, actualizar las credenciales de MySQL (username y password).
3. **Gestión de Recursos Visuales:**

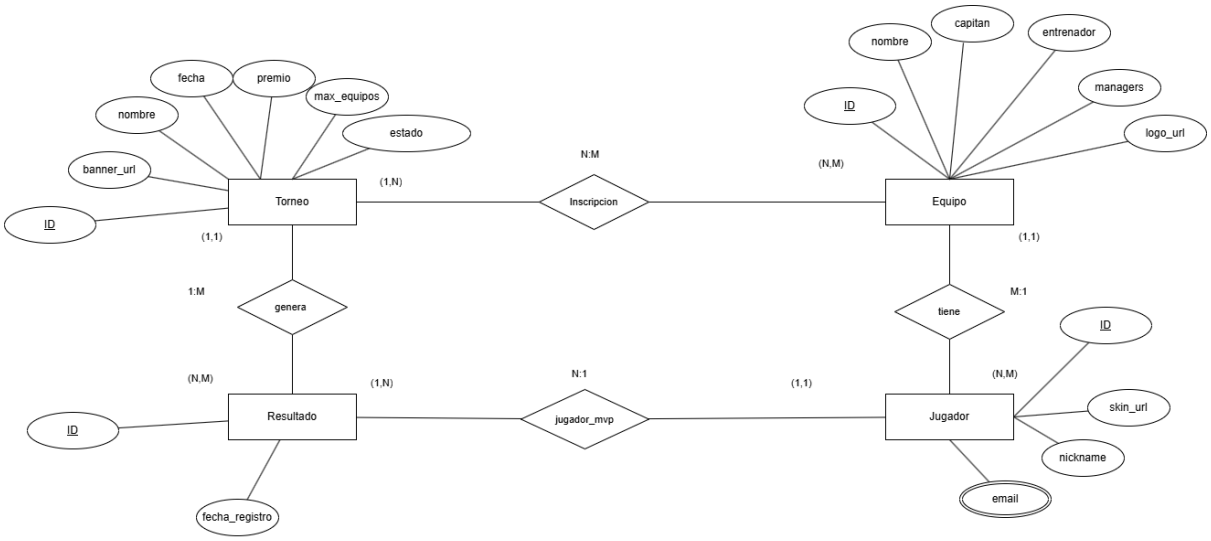
   a. Asegurarse de que las carpetas /images/banners/, /images/logos/ y /images/skins/ dentro de static contengan los archivos .png o .jpg referenciados en la base de datos.
4. **Ejecución:**
   a. Ejecutar la clase principal ZaloriaApplication.java.
   b. Acceder a http://localhost:8080 para la vista pública o http://localhost:8080/admin para la gestión.

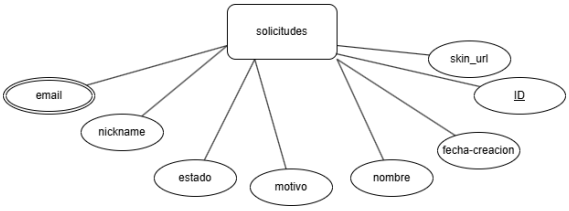
Puntos clave

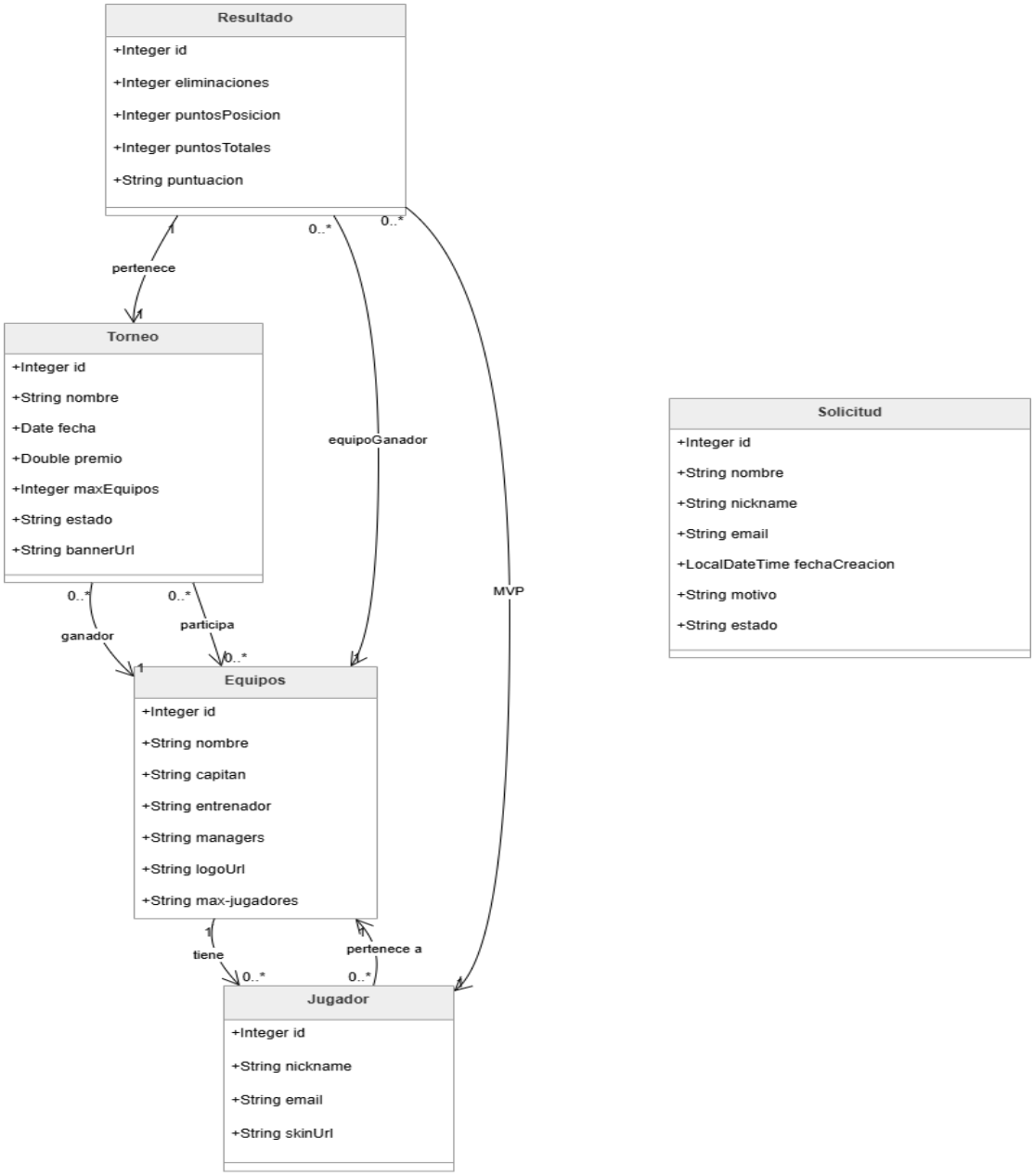
- **Punto Clave 1:** La relación ManyToMany entre Torneo y Equipos permite que la competición sea dinámica.
- **Punto Clave 2:** La simulación de resultados actualiza automáticamente el estado del torneo a FINALIZADO y dispara la actualización de los rankings públicos.
- **Punto Clave 3:** El uso de DTOs en el repositorio optimiza el rendimiento al no traer objetos completos cuando solo necesitamos nombres y conteos para el Ranking.

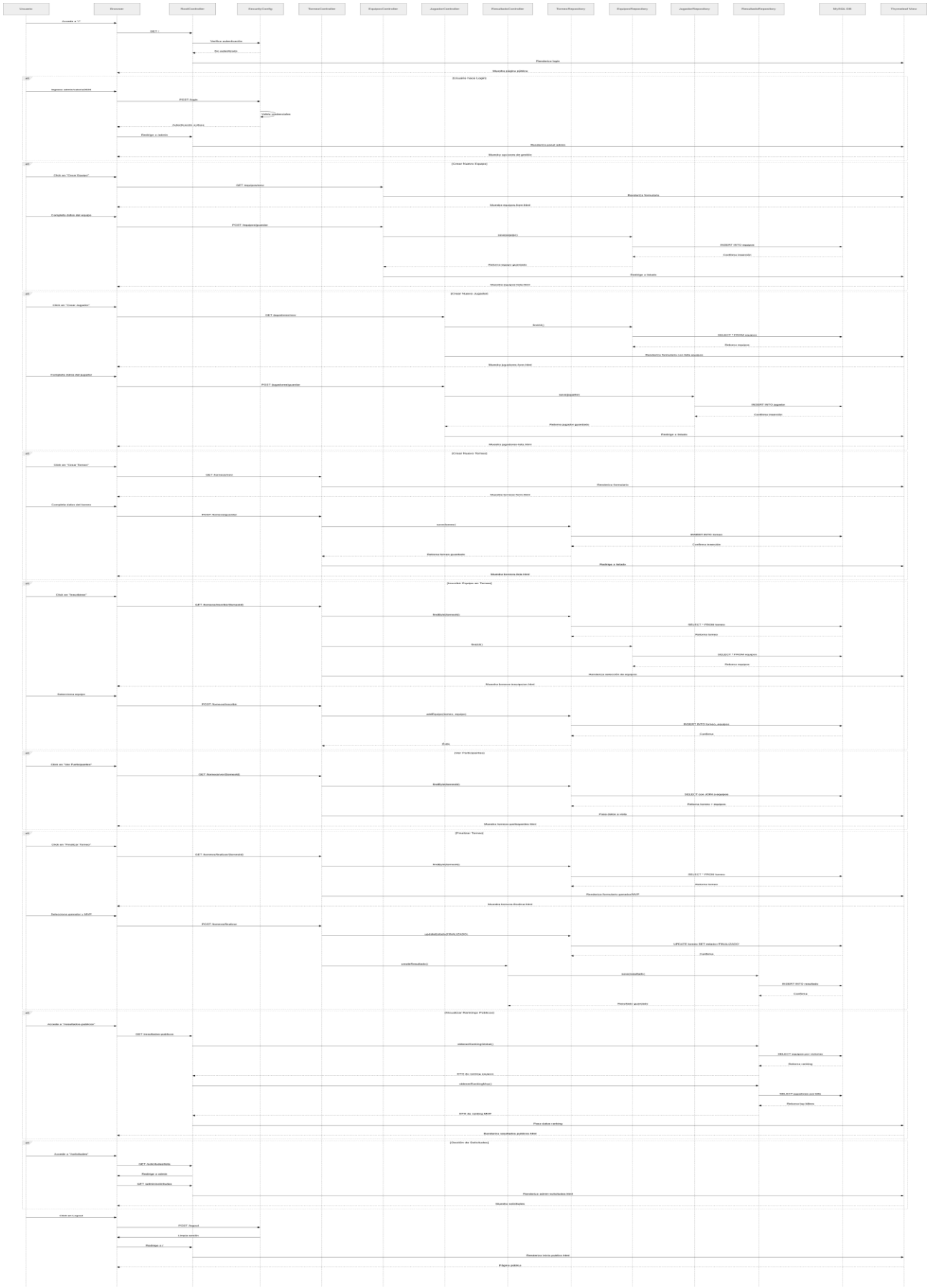
Anexos Técnicos (Diagramas)








Resultado
+Integer id
+Integer eliminaciones
+Integer puntosPosicion
+Integer puntosTotales
+String puntuacion
1
pertenece
1
0..*
0..*
Torneo
+Integer id
+String nombre
+Date fecha
+Double premio
+Integer maxEquipos
+String estado
+String bannerUrl
equipoGanador
MVP
Solicitud
+Integer id
+String nombre
+String nickname
+String email
+LocalDateTime fechaCreacion
+String motivo
+String estado
0..*
0..*
ganador
participa
1
0..*
Equipos
+Integer id
+String nombre
+String capitan
+String entrenador
+String managers
+String logoUrl
+String max-jugadores
1
tiene
0..*
1
pertenece a
0..*
Jugador
+Integer id
+String nickname
+String email
+String skinUrl


Usuario
Browser
FrontController
SecurityConfig
TorneoController
EquiposController
JugadorController
ResultadosController
TorneoRepository
EquipoRepository
JugadorRepository
ResultadoRepository
MySQL DB
Thymeleaf View
Accede a "/"
GET /
Verifica autenticación
No autenticado
Renderiza login
Muestra página pública
alt
[Usuario hace Login]
Ingresa admin/contraseña
POST /login
valida credenciales
Autenticación exitosa
Redirige a /admin
Renderiza panel admin
Muestra opciones de gestión
alt
[Crear Nuevo Equipo]
Click en "Crear Equipo"
GET /equipos/nuevo
Renderiza formulario
Muestra equipos-form.html
Completa datos del equipo
POST /equipos/guardar
save(equipo)
INSERT INTO equipos
Confirma inserción
Retorna equipo guardado
Redirige a /admin
Muestra equipos-lista.html
alt
[Crear Nuevo Jugador]
Click en "Crear Jugador"
GET /jugadores/nuevo
findAll()
SELECT * FROM equipos
Retorna equipos
Renderiza formulario con lista equipos
Muestra jugadores-form.html
Completa datos del jugador
POST /jugadores/guardar
save(jugador)
INSERT INTO jugador
Confirma inserción
Retorna jugador guardado
Redirige a /admin
Muestra jugadores-lista.html
alt
[Crear Nuevo Torneo]
Click en "Crear Torneo"
GET /torneos/nuevo
Renderiza formulario
Muestra torneos-form.html
Completa datos del torneo
POST /torneos/guardar
save(torneo)
INSERT INTO torneo
Confirma inserción
Retorna torneo guardado
Redirige a /admin
Muestra torneos-lista.html
alt
[Inscribir Equipo en Torneo]
Click en "Inscribirse"
GET /torneos/inscribir/{torneoId}
findById(torneoId)
SELECT * FROM torneo
Retorna torneo
findAll()
SELECT * FROM equipos
Retorna equipos
Renderiza selección de equipo
Muestra torneo-inscripcion.html
Selecciona equipo
POST /torneos/inscribir
addEquipo(torneo, equipo)
INSERT INTO torneo_equipo
Confirma
Ok
alt
[Ver Participantes]
Click en "Ver Participantes"
GET /torneos/ver/{torneoId}
findById(torneoId)
SELECT con JOIN a equipos
Retorna torneo + equipos
Pasa datos a vista
Muestra torneo-participantes.html
alt
[Finalizar Torneo]
Click en "Finalizar Torneo"
GET /torneos/finalizar/{torneoId}
findById(torneoId)
SELECT * FROM torneo
Retorna torneo
Renderiza formulario ganador/MVP
Muestra torneo-finalizar.html
Selecciona ganador y MVP
POST /torneos/finalizar
updateEstado(FINALIZADO)
UPDATE torneo SET estado='FINALIZADO'
Confirma
saveAll(resultados)
saveAll(resultados)
INSERT INTO resultado
Confirma
Resultado guardado
alt
[Visualizar Rankings Públicos]
Accede a "/resultados-publicos"
GET /resultados-publicos
obtenerRankingGlobal()
SELECT equipos por victorias
Retorna ranking
DTO de ranking equipos
obtenerRankingMvp()
SELECT jugadores por kills
Retorna top killers
DTO de ranking MVP
Pasa datos ranking
Renderiza resultados-publicos.html
alt
[Gestión de Solicitudes]
Accede a "/solicitudes"
GET /solicitudes/lista
Redirige a admin
GET /admin/solicitudes
Renderiza admin-solicitudes.html
Muestra solicitudes
Click en Logout
POST /logout
Limpia sesión
Redirige a /
Renderiza inicio.public.html
Página pública